
COMPTE RENDU



Mini Projet :

Java Avancée / Architectures Logicielles...

Module : **Java Avancée & Architectures Logicielles...**

Encadrant : **Mme. Wafa Neji Stancati & Mme. Hela Jabbar...**

Année universitaire : **2025 / 2026**

Réaliser par : **kamar zarrouki.**

Ichrak silini.

Raoua abdelwehed.

Emna naimi.

1. Introduction :

Ce mini-projet a été réalisé dans le cadre des modules Java Avancée et Architectures Logicielles a l'ISG Bizerte. Il consiste à concevoir et développer un micro service complet en utilisant Spring Boot et Spring Data JPA.

2.Objectifs :

- Concevoir un micro service avec Spring Boot.
- Utiliser Spring Data JPA pour la persistance des données.
- Mettre en place une architecture en couches : Controller, Service, Repository, Model.
- Tester les différents types de mapping JPA : OneToOne, OneToMany, ManyToMany.
- Exposer des API REST et les tester avec Postman.

3. Technologies utilisées :

Technologie	Version	Rôle
Spring Boot	4.0.5	Framework principal du micro service
Spring Data JPA	4.0.4	Persistance et accès aux données
Hibernate	7.2.7	ORM - Mapping Objet/Relationnel
PostgreSQL	42.7.10	Système de gestion de base de données
Lombok	1.18.44	Génération automatique getters/setters
Jackson	2.19+	Sérialisation/Désérialisation JSON
Postman	Dernière	Test des API REST
IntelliJ IDEA	2025.3.2	Environnement de développement
Java	25.0.2	Langage de programmation
PgAdmin	Dernière	Administration de la base de données

4. Entités et tables :

Entité Java	Table BD	Description
School	t_school	Etablissement scolaire
Departement	t_department	Departement de l'école
Student	t_student	Etudiant inscrit dans l'école
Adresse	t_address	Adresse de l'étudiant
Instructor	t_instructor	Enseignant de l'école
Course	t_course	Cours enseigné par l'instructeur
(Table jointure)	t_instructor_t_course	Jointure ManyToMany Instructor-Course

Question 2 - Mapping School <-> Department (OneToMany)

⇒ Une école contient plusieurs départements.

CAS 1 : Unidirectionnel (@OneToMany simple)

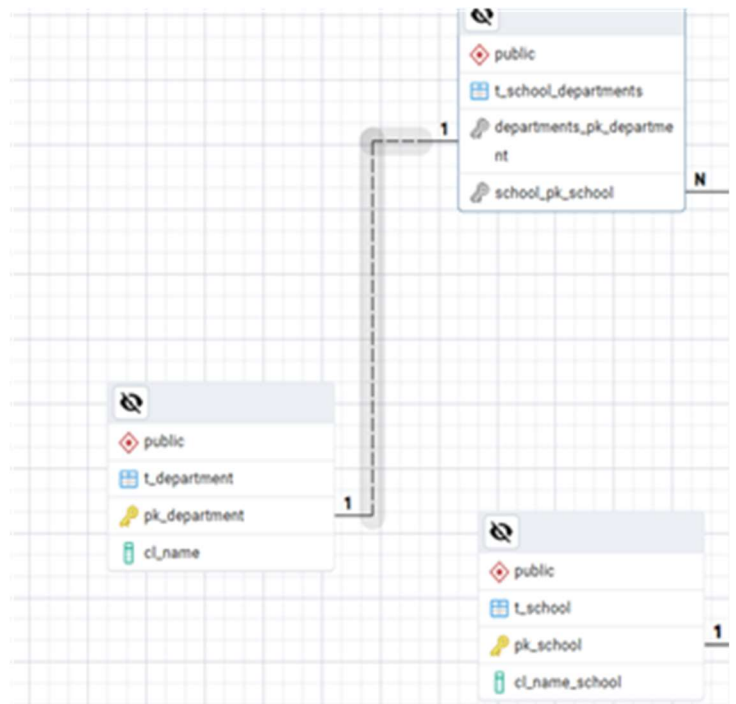
Code School.java :

```
@OneToMany(cascade = CascadeType.ALL)
private List<Departement> departments = new ArrayList<>();
```

Code Departement.java :

```
// Pas de @ManyToOne - relation gérée par @JoinTable dans School
```

- **Code** : Utilisation de `@OneToMany(cascade = CascadeType.ALL)` dans `School.java` sans référence inverse dans `Departement.java`.
- **Schéma** : Hibernate génère trois tables : `t_school`, `t_department` et une table de jointure `t_school_departments`.
- **Navigation** : Uniquement possible de `School` vers ses `Departments`.
- **Inconvénient** : La création d'une table intermédiaire est inutile pour une relation One-to-Many simple



CAS 2 : Bidirectionnel (Sans mappedBy)

Code School.java :

```
@OneToMany(cascade = CascadeType.ALL)
@JsonIgnoreProperties({"school", "hibernateLazyInitializer",
"handler"})
private List<Departement> departments = new ArrayList<>()
```

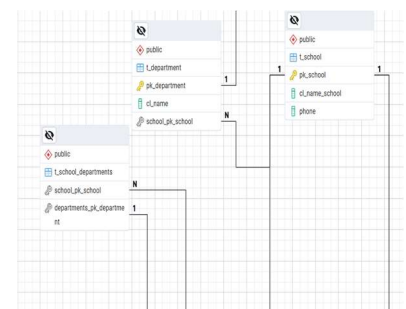
Code Departement.java :

```
@ManyToOne(fetch = FetchType.LAZY)
```

```
@JoinColumn(name = "school_PK_school")
```

```
private School school;
```

- **Code** : `@OneToMany` dans `School` et `@ManyToOne` avec `@JoinColumn` dans `Departement`.

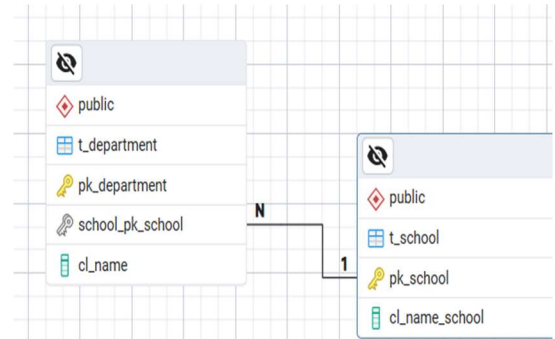


- **Schéma** : Bien que la clé étrangère school_PK_school soit définie, Hibernate maintient l'usage d'une table de jointure pour assurer la cohérence bidirectionnelle.
- **Navigation** : Bidirectionnelle (School → Departments ET Department → School).
- **Résultat** : Le schéma reste identique au Cas 1 (présence d'une table de jointure), mais permet la navigation dans les deux sens.

CAS 3 : Department -> School...

Code School.java :

```
@OneToMany(mappedBy = "school", cascade =
CascadeType.ALL, fetch = FetchType.LAZY)
@JsonIgnoreProperties({ "school", "hibernateLazyInitializer",
"handler" })
private List<Departement> departments = new ArrayList<>();
```



Code Departement.java :

```
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "school_PK_school")
private School school;
```

- **Code** : L'attribut mappedBy = "school" est ajouté dans School.java pour indiquer que le côté Department possède la relation.
- **Schéma** : Seules deux tables sont créées (t_school et t_department).
- **Clé Étrangère** : La FK school_PK_school est stockée directement dans la table t_department.

Justification du choix - Cas 3

Le Cas 3 est le plus adéquat car :

il est le plus performant et respecte les standards relationnels. En utilisant mappedBy du côté School et @JoinColumn du côté Department, on évite la création d'une table de jointure superflue, ce qui simplifie les requêtes SQL et optimise le stockage

Question 4 - Mapping Student <-> Adresse (OneToOne)

⇒ La relation entre Student et Adresse est OneToOne : chaque étudiant possède une seule adresse.

CAS 1 - Unidirectionnel avec FK dans t_address...

Code Student.java :

```
@OneToOne(cascade = CascadeType.ALL)
```

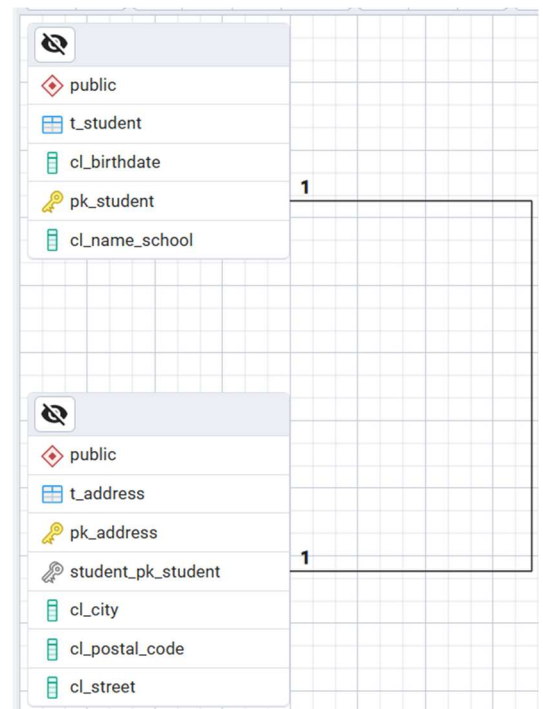
```
@JoinColumn (name = "student_PK_Student") // FK dans t_address
```

```
private Adresse adresse;
```

Code Adresse.java :

```
// Pas de relation vers Student
```

- FK student_PK_Student placée dans t_address
- Navigation uniquement de Student vers Adresse



CAS 2 - Bidirectionnel avec FK dans t_address

Code Student.java :

```
@OneToOne(cascade = CascadeType.ALL)
```

```
@JoinColumn(name = "student_PK_Student")
```

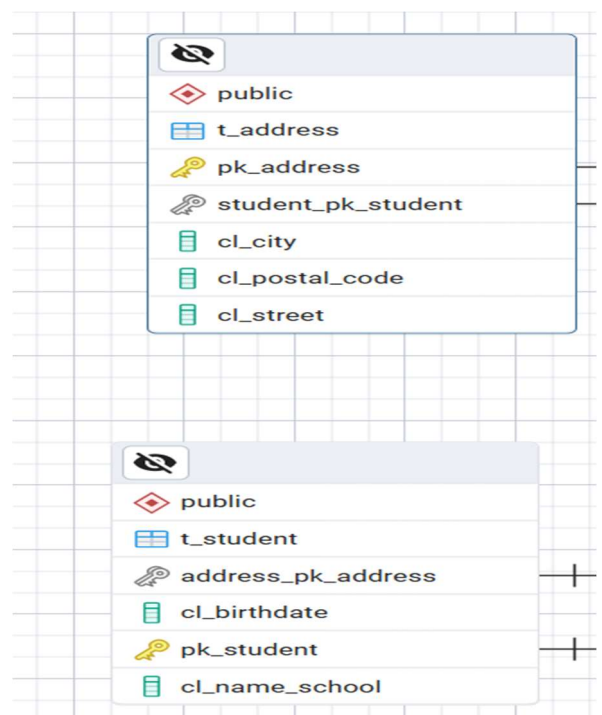
```
private Adresse adresse;
```

Code Adresse.java :

```
@OneToOne(mappedBy = "adresse")
```

```
private Student student;
```

- FK student_PK_Student dans t_address
- Navigation bidirectionnelle des deux cotes



CAS 3 : Bidirectionnel avec FK dans t_student (MEILLEUR)...

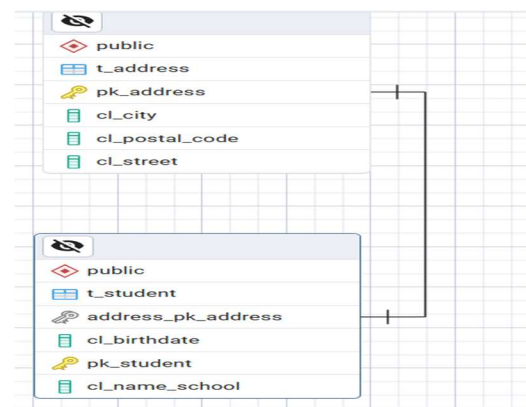
Code Student.java :

```
@OneToOne(cascade = CascadeType.ALL)
```

```
@JoinColumn(name = "address_PK_ADDRESS") // FK dans t_student
```

```
private Adresse adresse;
```

- FK address_PK_ADDRESS placée dans t_student (entité principale)



- Correspond exactement au schéma final Figure 5 du projet

Justification du choix - Cas 3:

Le Cas 3 est le plus adéquat car :

- Student est propriétaire => FK address_PK_ADDRESS dans t_student (sémantiquement correct)
- Navigation bidirectionnelle possible : Student connaît son adresse et Adresse connaît son étudiant
- Schéma conforme à la Figure 5 du sujet
- Le Cas 1 est contre-intuitif (FK dans t_address), le Cas 2 est unidirectionnel et limite

Question 6 : Mapping Instructor <-> Course (ManyToMany)

⇒ La relation entre Instructor et Course est ManyToMany : un instructeur peut enseigner plusieurs cours, et un cours peut être enseigné par plusieurs instructeurs.

CAS 1 : Bidirectionnel, Instructor propriétaire...

Code Instructor.java :

```
@ManyToMany(cascade = {CascadeType.PERSIST,
CascadeType.MERGE})
```

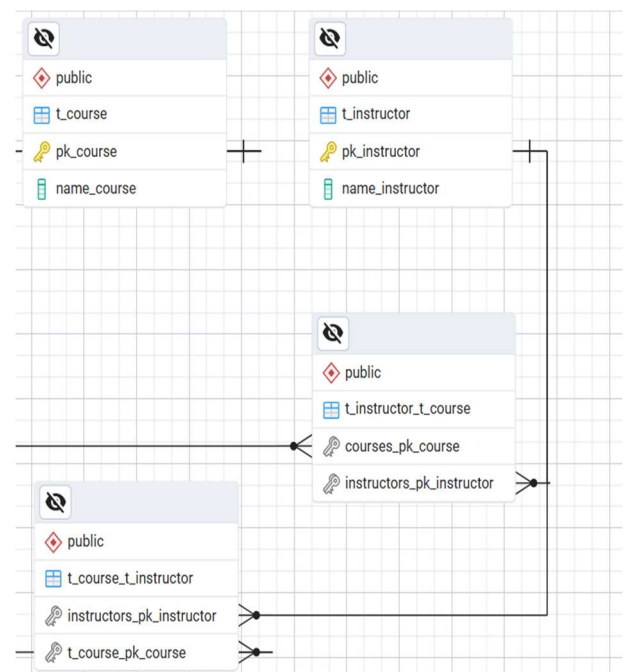
```
@JoinTable(
    name = "t_course_t_instructor",
    joinColumns = @JoinColumn(name =
"instructors_pk_instructor"),
    inverseJoinColumns = @JoinColumn(name =
"t_course_pk_course")
)
```

```
private List<Course> courses;
```

Code Course.java :

```
@ManyToMany(mappedBy = "courses")
```

```
private List<Instructor> instructors;
```



CAS 2 : Bidirectionnel, Course propriétaire (MEILLEUR)...

Code Instructor.java :

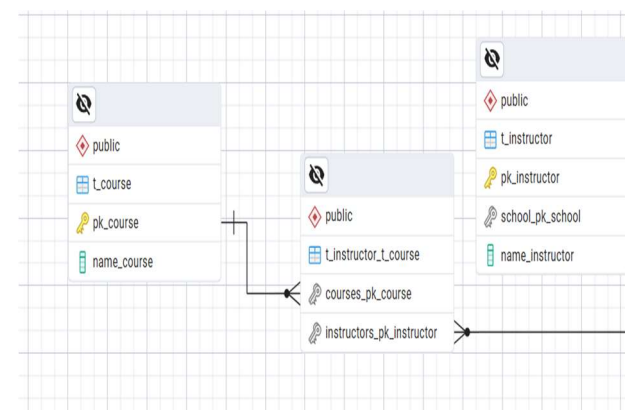
```
@ManyToMany(mappedBy = "instructors")
```

```
private List<Course> courses;
```

Code Course.java :

```
@ManyToMany(cascade = {CascadeType.PERSIST,
CascadeType.MERGE})
```

```
@JoinTable (name = "t_instructor_t_course",
```



```

joinColumns = @JoinColumn(name = "courses_pk_course"),
inverseJoinColumns = @JoinColumn(name = "instructors_pk_instructor")
)

```

```
private List<Instructor> instructors;
```

- Une seule table de jointure : t_instructor_t_course
- Colonnes : courses_pk_course + instructors_pk_instructor
- Correspond exactement au schéma final Figure 5

Justification du choix - Cas 2 :

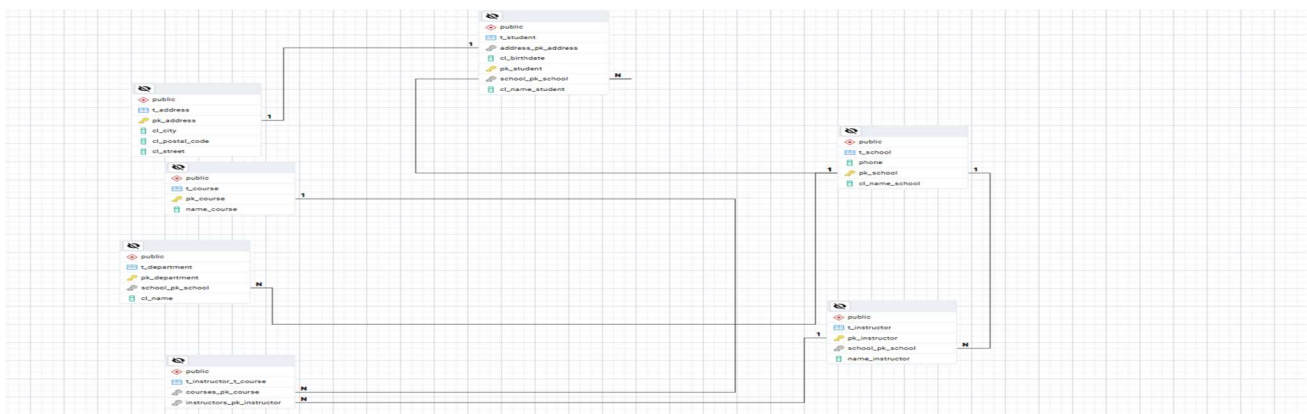
- La table de jointure t_instructor_t_course est conforme au schéma final de la Figure 5
- Instructor est le cote naturellement propriétaire car il gère ses cours
- Navigation bidirectionnelle : Instructor.getCourses() et Course.getInstructors()

Question 7 - Mapping Final Complet

⇒ Le mapping final respecte toutes les règles de navigabilité définies dans le sujet :

Relation	Type	Direction	Annotation clé
Student -> Adresse	OneToOne	Unidirectionnel	@JoinColumn(name="address_PK_ADDRESS")
School <-> Department	OneToMany/ManyToOne	Bidirectionnel	mappedBy="school" + @JoinColumn
Instructor <-> Course	ManyToMany	Bidirectionnel	@JoinTable(t_instructor_t_course)
Instructor <-> School	ManyToOne/OneToMany	Bidirectionnel	mappedBy="school" + @JoinColumn
Student <-> School	ManyToOne/OneToMany	Bidirectionnel	mappedBy="school" + @JoinColumn

Schéma final de la base de données :



Question 9 - TestSchoolController

Exemples de tests Postman

a. Créer une nouvelle School avec une liste de Student, d' Instructor et de Department

POST

<http://localhost:8080/api/test/school>

```
{
  "name": "ISGB",
  "phone": 72431000,
  "departments": [
    { "name": "Informatique" },
    { "name": "Gestion" }
  ],
  "students": [
    {
      "name": "kamar zarrouki",
      "birthDate": "2005-05-02",
      "adresse": {
        "street": "5 Avenue Habib Bourguiba",
        "city": "Bizerte",
        "postalCode": "7000"
      }
    }
  ],
  "instructors": [
    { "name": "Prof. Neji" }
  ]
}
```

b. Retourner une School en se basant sur son id

GET

<http://localhost:8080/api/test/school/1>

c. Créer un nouveau Student avec son Address et son School

POST

<http://localhost:8080/api/test/student/1>

```
{
  "name": "ichrak silini",
  "birthDate": "2003-03-08",
  "adresse": {
    "street": "Rue Ibn Khaldoun",
    "city": "Tunis",
    "postalCode": "1000"
  }
}
```

d. Lister tous les Students.

GET

<http://localhost:8080/api/test/students>

e. Créer un Instructor avec une liste de cours

POST

<http://localhost:8080/api/test/instructor/1>

```
{
  "name": "Prof. Jabbar",
  "courses": [
    { "name": "Architectures Logicielles" },
    { "name": "programmation web" }
  ]
}
```

f. Lister les Instructors par nom.

GET

<http://localhost:8080/api/test/instructor/search?name=Prof. Jabbar>

g. Retourner un Instructor par id.

GET

<http://localhost:8080/api/test/instructor/1>

h. Retourner un Course par id.

GET

<http://localhost:8080/api/test/course/1>

i. Lister les Courses d'un Instructor en se basant sur son id.

GET

<http://localhost:8080/api/test/instructor/1/courses>

j. Rajouter un nouveau Course à un Instructor existant

POST

<http://localhost:8080/api/test/instructor/1/course>

```
{  
  "name": "java"  
}
```

Conclusion

Ce mini-projet a concrétisé les concepts de **Java Avancé** et des **Architectures Logicielles**. Les points clés retenus sont :

- **Navigabilité** : Le choix de l'unidirectionnel (notamment pour *Student* → *Adresse*) simplifie le schéma SQL et évite les boucles de récursion JSON.
- **Modélisation Métier** : Définir l'*Instructor* comme propriétaire de la relation ManyToMany assure une cohérence logique avec les processus réels.
- **Fiabilité** : L'usage de `@Transactional` s'est avéré crucial pour la gestion du chargement LAZY et l'intégrité des données.
- **Structure** : L'architecture en couches (Controller, Service, Repository) garantit un code évolutif et une séparation claire des responsabilités.

L'application est pleinement opérationnelle : les dix méthodes du service sont implémentées, exposées via des API REST et validées avec succès sur Postman.